

Hardness of learning abelian loops in sublinear time

Yu-Han Cheng and Ching-Lueh Chang

Department of Computer Science and Engineering (both authors)
Innovation Center for Big Data and Digital Convergence (Ching-Lueh Chang)

Yuan Ze University, Taoyuan, Taiwan

s1016043@mail.yzu.edu.tw (Yu-Han Cheng)

clchang@saturn.yzu.edu.tw (Ching-Lueh Chang)

Abstract

Chang shows that each randomized $o(|G|^2)$ -query algorithm can recover only an expected $o(1)$ fraction of the multiplication table of some finite abelian loop $(G, \cdot)$, where both multiplication and inversion queries are allowed. However, he leaves open whether there exists one computable family of abelian loops hard for all $o(|G|^2)$ -query algorithms to recover, where only multiplication queries are allowed. The answer is yes, and we will solve this problem in this paper.

1 Introduction

Many properties of a group-like structure can be discovered from its multiplication table. Now we are interested in how many queries we need for the exact recovery of the multiplication table of any finite group-like structure. We can reduce the number of queries for recovering the multiplication table of any finite group-like structure by its structural axioms. Zumbrägel et al. [2] present an $O(|G|)$ -query algorithm for recovering the Cayley table of an abelian group. Chang [3] shows that each randomized $o(|G|^2)$ -query algorithm can recover only an expected $o(1)$ fraction of the multiplication table of some finite abelian loop $(G, \cdot)$, where both multiplication and inversion queries are allowed. Furthermore, Chang [3] also shows that each randomized $o(|G|^2)$ -query algorithm can recover only an expected $o(1)$ fraction of any of the multiplication tables of some finite commutative semiring $(R, +, \cdot)$ with $(R, +)$ being a commutative aperiodic monoid.

Chang leaves open the following question: Does there exist one computable family of abelian loops hard for all $o(|G|^2)$ -query algorithms to recover (where each query may ask for the product of two elements in G)? Later, we will answer this question in the affirmative by actually constructing one such family of abelian loops.

2 Preliminaries

We refer to some basic definitions in algebra [1].

Definition 1. A nonempty set G endowed with a binary operation $\cdot : G \times G \rightarrow G$ is called a groupoid.

Definition 2. An element $e \in G$ is an identity if and only if for all $x \in G$, $x \cdot e = e \cdot x = x$.

Definition 3. An element $x \in G$ is an inverse of $y \in G$ if $x \cdot y = y \cdot x = e$. If x is the unique inverse of y , then it is denoted y^{-1} .

Definition 4. A groupoid $(G, \cdot)$ is abelian if $x \cdot y = y \cdot x$ for all $x, y \in G$.

Definition 5. A groupoid $(G, \cdot)$ is a quasigroup if for all $x, y \in G$, there are unique elements $a, b \in G$ such that $x \cdot a = y$ and $b \cdot x = y$.

Definition 6. A groupoid $(G, \cdot)$ is a loop if $(G, \cdot)$ is a quasigroup with an identity.

The follow theorem is due to Chang.

Theorem 1. ([3]) For each randomized $o(G^2)$ -query algorithm A outputting a $|G|$ -by- $|G|$ multiplication table, there exists a family of finite abelian loops $(G, \cdot)$ such that the output of A with oracle access to $(G, \cdot)$ coincides with the multiplication table of $(G, \cdot)$ in an expected $o(1)$ fraction of entries, where the expectation is taken over the random coin tosses of A . Above, each query may ask for $x \cdot y$ or x^{-1} on any choice of $x, y \in G$.

Chang's paper does not answer if there exists a computable family of finite abelian loops fitting Theorem 1. The next section answers that.

3 Constructing a hard-to-learn abelian loop

In this section, we construct a family of abelian loops to fit Theorem 1.

Write $\{0,1\}^*$ for the set of all 0-1 strings, 0^* for the set of strings containing no symbols other than 0 and $\preceq$ for the lexicographical ordering on

strings. Strings are allowed to have length 0. For $s \in \{0,1\}^*$ with $|s| \geq k$, the k th bit of s is denoted by s_k .

Let $\mathbf{b}(s, t)$ be independent and uniformly random bits for all $s, t \in \{0,1\}^* \setminus 0^*$ with $s \leq t$. Then define $\mathbf{b}(s, t) = \mathbf{b}(t, s)$ for $s, t \in \{0,1\}^* \setminus 0^*$ with $t < s$. Finally, let $\mathbf{b}(0^k, s) = \mathbf{b}(s, 0^k) = 0$ for $s \in \{0,1\}^* \setminus 0^*$ and $k \in \mathbb{N}$.

Suppose that n is a power of 2. For $s, t \in \{0,1\}^{\lg n}$, let $s \diamond t$ be the string whose i th bit is

$$s_i \oplus t_i \oplus \mathbf{b}(s_1 s_2 \dots s_{i-1}, t_1 t_2 \dots t_{i-1}) \quad (1)$$

for $i \in \{1, 2, \dots, \lg n\}$, where $\oplus$ is the XOR operation.

Lemma 1. For all $s, t \in \{0,1\}^{\lg n}$, $s \diamond t \in \{0,1\}^{\lg n}$.

Proof. By equation (1),

$$(s \diamond t)_i = s_i \oplus t_i \oplus \mathbf{b}(s_1 s_2 \dots s_{i-1}, t_1 t_2 \dots t_{i-1}) \in \{0,1\}$$

for $i \in \{1, 2, \dots, \lg n\}$. So, $s \diamond t \in \{0,1\}^{\lg n}$. #

Lemma 2. For all $s, t \in \{0,1\}^{\lg n}$, $s \diamond t = t \diamond s$.

Proof. By definition, $\mathbf{b}(s, t) = \mathbf{b}(t, s)$ for $s, t \in \{0,1\}^{\lg n}$ with $t \leq s$. So,

$$\begin{aligned} (s \diamond t)_i &= s_i \oplus t_i \oplus \mathbf{b}(s_1 s_2 \dots s_{i-1}, t_1 t_2 \dots t_{i-1}) \\ &= t_i \oplus s_i \oplus \mathbf{b}(t_1 t_2 \dots t_{i-1}, s_1 s_2 \dots s_{i-1}) \\ &= (t \diamond s)_i \end{aligned}$$

for $i \in \{1, 2, \dots, \lg n\}$. So, $s \diamond t = t \diamond s$. #

Lemma 3. There is an element $e \in \{0,1\}^{\lg n}$ such that $s \diamond e = e \diamond s = s$ for all $s \in \{0,1\}^{\lg n}$.

Proof. Let $e = 0^{\lg n} \in \{0,1\}^{\lg n}$. Then

$$\begin{aligned} (s \diamond e)_i &= (e \diamond s)_i \text{ (by lemma 2)} \\ &= s_i \oplus e_i \oplus \mathbf{b}(s_1 s_2 \dots s_{i-1}, e_1 e_2 \dots e_{i-1}) \\ &= s_i \oplus 0 \oplus \mathbf{b}(s_1 s_2 \dots s_{i-1}, e_1 e_2 \dots e_{i-1}) \\ &= s_i \oplus \mathbf{b}(s_1 s_2 \dots s_{i-1}, 00 \dots 0) \\ &= s_i \oplus 0 \text{ (by equation (1))} \\ &= s_i \end{aligned}$$

for $i \in \{1, 2, \dots, \lg n\}$. So, there is a unique identity $e = 0^{\lg n} \in \{0,1\}^{\lg n}$ such that $s \diamond e = e \diamond s = s$ for all $s \in \{0,1\}^{\lg n}$. #

Lemma 4. For all $s, t \in \{0,1\}^{\lg n}$, there is a unique element $x \in \{0,1\}^{\lg n}$ such that $s \diamond x = t$.

Proof. By definition, $s \diamond x = t$ if and only if

$$s_i \oplus x_i \oplus \mathbf{b}(s_1 s_2 \dots s_{i-1}, x_1 x_2 \dots x_{i-1}) = t_i, \quad (2)$$

for $i \in \{1, 2, \dots, \lg n\}$.

We proceed to solve equation (2) for $i \in \{1, 2, \dots, \lg n\}$.

First, for $i = 1$:

$s_1 \oplus x_1 \oplus \mathbf{b}(\lambda, \lambda) = t_1$, then we can get the unique result x_1 ,

$$x_1 = s_1 \oplus \mathbf{b}(\lambda, \lambda) \oplus t_1,$$

where λ denotes the string with length zero (commonly referred to as the empty string).

Second, for $i = 2$:

$s_2 \oplus x_2 \oplus \mathbf{b}(s_1, x_1) = t_2$, then we can get the unique result

$$\begin{aligned} x_2 &= s_2 \oplus \mathbf{b}(s_1, x_1) \oplus t_2 \\ &\quad (x_1 \text{ is a unique result.}) \end{aligned}$$

Next, for $i = 3$:

$s_3 \oplus x_3 \oplus \mathbf{b}(s_1 s_2, x_1 x_2) = t_3$, then we can get the unique result

$$\begin{aligned} x_3 &= s_3 \oplus \mathbf{b}(s_1 s_2, x_1 x_2) \oplus t_3. \\ &\quad (x_1 \text{ and } x_2 \text{ are unique results.}) \end{aligned}$$

Suppose after continuing the steps, we get unique results x_i for $i \in \{1, 2, \dots, \lg n - 1\}$. Then we can get

$$x_{\lg n} =$$

$$s_{\lg n} \oplus \mathbf{b}(s_1 s_2 \dots s_{\lg n-1}, x_1 x_2 \dots x_{\lg n-1}) \oplus t_{\lg n}.$$

($x_1, x_2, \dots, x_{\lg n-1}$ are unique results.)

By definition, we can get that the element x that satisfies $s \diamond x = t$ is unique. #

Lemmas 1–4 yield the following.

Lemma 5. $(\{0,1\}^{\lg n}, \diamond)$ is an abelian loop with identity $e = 0^{\lg n}$.

Having constructed the abelian loop $(\{0,1\}^{\lg n}, \diamond)$, we begin to show that the loop we constructed fits Theorem 1. Before that, we say that (s, t) begins with (x, y) if (1) x is a prefix of s and y is a prefix of t or (2) y is a prefix of s and x is a prefix of t .

Lemma 6. Let ALG be an algorithm with oracle access to $(\{0,1\}^{\lg n}, \diamond)$, $k \in \{1, 2, \dots, \lg n - 1\}$, $s, t \in \{0,1\}^{\lg n} \setminus 0^*$ and

Q

$$= \{(u, v) \in \{0,1\}^{\lg n} \times \{0,1\}^{\lg n} \mid ALG \text{ ever queries for } u \diamond v\}.$$

Conditional on any realization of ALG 's output and on ALG not making any query that begins with $(s_1 s_2 \dots s_k, t_1 t_2 \dots t_k)$,

$(s \diamond t)_{k+1} (s \diamond t)_{k+2} \dots (s \diamond t)_{\lg n}$ distributes uniformly over $\{0,1\}^{\lg n-k}$.

Proof. By equation (1), we may think of a query for $u \diamond v$, where $(u, v) \in Q$, as returning

$$\mathbf{b}(u_1 u_2 \dots u_i, v_1 v_2 \dots v_i)$$

for $i \in \{0, 1, \dots, \lg n - 1\}$ instead of $u \diamond v$.

Observe that $\mathbf{b}(x, y)$ is returned if and only if there exists a query $(u, v) \in Q$ beginning with (x, y) , where $x, y \in \{0,1\}^* \setminus 0^*$ and $|x| = |y| \leq \lg n$.

Without loss of generality, we defer the picking of $\mathbf{b}(x, y)$ until it must be returned, where $x, y \in \{0,1\}^* \setminus 0^*$ and $|x| = |y| \leq \lg n$. When

$\mathbf{b}(x, y)$ must be returned, it is picked uniformly at random and independently from the bits already picked. The reason is that $\mathbf{b}(x, y)$ is a function which uniformly and randomly outputs a bit. Conditional on ALG not making any query that begins with $(s_1 s_2 \dots s_k, t_1 t_2 \dots t_k)$, therefore, all of $\mathbf{b}(s_1 s_2 \dots s_j, t_1 t_2 \dots t_j)$

with $j \in \{k, k+1, \dots, \lg n - 1\}$ will remain unpicked after all queries of ALG return; hence we may think of them as being picked uniformly at random after ALG outputs. By the above reason and equation (1), we get that

$(s \diamond t)_{k+1} (s \diamond t)_{k+2} \dots (s \diamond t)_{\lg n}$ distributes uniformly over $\{0, 1\}^{\lg n - k}$.

Theorem 2. Given oracle access to $(\{0, 1\}^{\lg n}, \diamond)$, the output of any $o(n^2)$ -query algorithm, interpreted as a multiplication table on $\{0, 1\}^{\lg n}$, coincides with the multiplication table of $(\{0, 1\}^{\lg n}, \diamond)$ on an expected $o(1)$ fraction of entries.

Proof. Let ALG be an algorithm with oracle access to $(\{0, 1\}^{\lg n}, \diamond)$ and query complexity $q(n) = o(n^2)$. Let

$$k = k(n) = \lg n - \lg \lg \left(\frac{n^2}{q(n)} \right).$$

For uniformly random $u, v \in \{0, 1\}^{\lg n}$, it is not hard to see that

$$\Pr[\text{None of ALG's queries begins with } (u_1 \dots u_k, v_1 \dots v_k)] = 1 - o(1)$$

by a counting argument. Furthermore, conditional on any realization of ALG's outputs and on ALG

not making queries beginning with $(u_1 u_2 \dots u_k, v_1 v_2 \dots v_k)$, we have that

$$(u \diamond v)_{k+1} (u \diamond v)_{k+2} \dots (u \diamond v)_{\lg n}$$

distributes uniformly over $\{0, 1\}^{\lg n - k}$ by Lemma 6, allowing $u \diamond v$ to equal the corresponding entry of ALG's output only with probability $1/2^{\lg n - k} = o(1)$. #

Acknowledgments

Ching-Lueh Chang is supported in part by the National Science Council under grant NSC101-2221-E-155-015-MY2 and the Ministry of Economic Affairs under grant 102-E0616.

References

- [1] D. Dummit and R. M. Foote. *Abstract Algebra*. Wiley, 3rd edition, 2003.
- [2] J. Zumbärgel, G. Maze, and J. Rosenthal. Efficient recovering of operation tables of black box groups and rings. In *Proceedings of the IEEE International Symposium on Information Theory*, pages 639-643, Toronto, Ontario, Canada, 2008.
- [3] C. L. Chang. Hardness of learning loops, monoids and semirings. *Discrete Applied Mathematics*, 162(1):149-158, 2014.